ClearURL: Explainable AI-Based Phishing URL Detection System

Technical Documentation

FIT5238 Group 15

August, 2026

System Technical Documentation

Contents

1 Introduction 2

2 System Architecture 2

3 Functional Design 3

4 AI Model Integration 5

5 Backend Design 8

6 Persistence Design 9

7 Frontend Design 10

8 Testing and Validation 11

9 Deployment and Runtime 11

10 Limitations and Future Improvements 12

11 Conclusion 12

1 Introduction

ClearURL is a full-stack phishing URL detection system designed for SME employees and security administrators. It combines an authenticated React frontend with a Python backend that validates URLs, retrieves webpage evidence when possible, extracts structured features, applies a trained machine-learning model, produces URL-specific explanations, and records the result for operational review.

The current system provides account registration for both supported roles, CAPTCHA verification, recovery-code password recovery, verification-status reporting, brand-impersonation analysis, administrator case decisions, policy overrides, account suspension, and paginated administration views. These capabilities operate together with the end-to-end employee analysis workflow to provide a complete phishing URL detection and review system.

1.1 Deployed Access URL

The current deployed prototype is accessible at:

`http://120.26.118.110/`

This hosted URL is the preferred demonstration route. The server environment must provide the backend process, reverse proxy, model artifact, and SQLite runtime files. If the server cannot reach external websites, the application still performs URL-only analysis and explicitly reports the reduced verification status.

1.2 System Scope

The current release provides:

- role-based authentication for Employee and Admin accounts
- CAPTCHA-protected sign-in, registration, password change, and password reset
- recovery-code based account recovery without an email dependency
- URL normalisation, including automatic HTTPS for a domain-only input
- live webpage HTML and redirect-chain retrieval when network access permits
- lexical, webpage, brand identity, and sensitive-path feature extraction
- XGBoost prediction with risk guardrails and an internal safety-score representation
- local feature-contribution based explanations and verification status
- employee risk reports and requests for review of possible misclassification
- administrator case resolution with reason and optional screenshot evidence
- persistent or time-limited policy entries and policy-result overrides
- brand impersonation visualisation and model quality monitoring
- account suspension without deletion of historical records
- server-side filtering and pagination for large administration datasets

2 System Architecture

ClearURL follows a layered client-server architecture:

- **Presentation layer**: React, TypeScript, and Vite

- **Application layer**: Starlette HTTP routes and role checks
- **Analysis layer**: preprocessing, webpage retrieval, feature extraction, prediction, and explanation
- **Feedback and policy layer**: employee reports, case review, allowlist/blocklist decisions, and policy lookup
- **Persistence layer**: separate SQLite databases for authentication, scans, feedback, and model records
- **Model artifact layer**: a serialized XGBoost bundle stored under `artifacts/`

The frontend communicates with the backend through JSON endpoints. The backend is the integration point for authentication, analysis, policy lookup, logging, feedback persistence, and administration. The code is separated by responsibility, but the deployed product runs as one frontend and one backend service.

2.1 Runtime Analysis Sequence

1. An authenticated employee enters a complete URL or a domain-only value.
2. The frontend submits the raw input to `POST /api/analyze`.
3. The backend validates and normalises the URL. If no scheme is supplied, `https://` is added.
4. The feedback repository is checked for an active administrator-reviewed policy entry before model inference.
5. If no policy override applies, the backend attempts webpage retrieval and follows redirects within the configured fetch policy.
6. URL identity and webpage features are extracted in a fixed order.
7. The model produces a phishing probability and local feature contributions when supported.
8. Model guardrails adjust risk when evidence is incomplete, strongly benign, or strongly indicative of phishing.
9. The explanation service maps relevant technical signals to employee-readable reasons and recommendations.
10. The scan is recorded in `prediction_logs.sqlite3`.
11. The API returns prediction, verification, explanation, and frontend fields in one response contract.

2.2 Architectural Rationale

The project uses one integrated backend service to reduce deployment and operational complexity while the prototype is being assessed. Internal separation allows individual responsibilities to be tested independently. SQLite is used because the project requires a portable demonstration deployment with separate data concerns rather than a shared database server.

3 Functional Design

3.1 Authentication and Account Security

Authentication uses role-based sessions. The supported roles are `employee` and `admin`. The registration form allows the user to select either role, and the backend stores the selected role

with the account. Administrative endpoints verify both a valid session and the `admin` role.

Passwords are stored as PBKDF2-SHA256 hashes. Sessions have an expiry period and are stored separately from user credentials. A CAPTCHA challenge is generated by `GET /api/auth/captcha`; the challenge is short-lived and must be answered for sign-in, registration, password changes, and password resets.

Registration returns a recovery code once. The database stores a hash of that code, not the displayed value. Change-password requires the account email, current password, new password, recovery code, and CAPTCHA. Forgot-password requires the email, recovery code, new password, and CAPTCHA. This implementation provides a local recovery mechanism without depending on an external mail server.

Administrator account management allows an admin to suspend an active employee account. Suspension sets the account inactive and prevents future authentication, but does not delete the account or its historical scan and feedback records. Admin accounts are protected from this employee-suspension action.

3.2 Employee URL Analysis

The employee enters a URL in the analysis workspace. A domain-only value such as `google.com` is accepted and normalised to an HTTPS URL. The response contains the normalised URL, preprocessing warnings, extracted features, model prediction, verification status, explanation reasons, and recommended actions.

The API calculates a safety score by inverting the final risk score: a higher value represents lower detected risk. The current employee interface deliberately does not display a numeric score. It presents the `Safe`, `Suspicious`, or `High Risk` category, verification status, explanations, and recommended actions. Risk and safety values remain available in the API payload, prediction log, scan filters, and brand-visualisation records for administrative use.

3.3 Webpage and Redirect Evidence

The backend attempts to retrieve the submitted page before feature extraction. The fetch service records the final URL, redirect chain, HTML availability, and retrieval warnings. The webpage parser extracts titles, forms, input fields, password fields, external form actions, iframes, scripts, external links, meta refresh indicators, JavaScript redirects, and domain-title consistency.

The response includes a verification status so that an inaccessible page is not silently treated as confirmed malicious. Possible statuses include webpage content verified, official domain recognised, short-link destination not verified, destination could not be verified, webpage access failed, limited webpage verification, and URL-only analysis.

3.4 Employee Feedback and Review Requests

After an analysis, an employee may open the review form. The employee can either confirm and report perceived risk or request a review of a possible misclassification. The form records the employee assessment, a structured reason code, and optional free-text notes. Reports are linked to the analysis request and aggregated into a review case so repeated submissions for the same normalised URL do not create uncontrolled duplicate cases.

3.5 Administrator Review and Policy Decisions

Administrators can inspect all review cases, filter them by status, risk category, review type, decision state, or URL, and process an individual case in a modal workflow. A decision requires a reason. A screenshot can be attached as evidence and is stored with the case metadata.

The administrator may resolve a case as:

- **allowlist**: the reviewed URL is treated as Safe by a policy override; the entry may be permanent or time-limited
- **blocklist**: the reviewed URL is treated as High Risk by a persistent policy override

The terminology represents reviewed policy decisions rather than an assertion that every unreachable or unrecognised domain is malicious. Policy lookup occurs before normal model inference, and the response identifies the override through its verification status and policy model version.

3.6 Administration Views

The admin console contains separate views for overview metrics, review cases, scan records, brand visualisation, policy lists, model monitoring, and account management. The following views load data from backend APIs rather than frontend placeholders:

- overview metrics and risk distribution
- review cases with ten records per page
- scan records with forty records per page
- brand visualisation with ten records per page
- policy entries with ten records per page
- registered accounts with server-side pagination
- model quality, recorded versions, and live operational signals

Filtering is applied before pagination on the backend. Therefore, the page count and displayed page contents reflect the filtered dataset rather than the unfiltered total. Pagination controls provide first-page, previous-page, next-page, and last-page actions.

4 AI Model Integration

4.1 Training Data and Preparation

The active training workflow uses a prepared PhiUSIIL feature dataset and additional contextual samples:

- `examples/phiusiil_model_features.csv`
- `examples/hard_negative_safe_samples.csv`
- `examples/official_safe_sites.csv`
- `examples/brand_impersonation_samples.csv`
- `examples/acceptance_context_samples.csv`

The current training report records dataset version `phiusiil-v9-official-domain-guardrails`, 166,289 cleaned rows, and 24 selected model features. The additional samples provide examples for official domains, hard negative safe URLs, brand impersonation, and acceptance-context cases. They complement the main dataset; they are not inserted as hard-coded prediction outputs.

4.2 Model Selection

The training pipeline compares Logistic Regression, Random Forest, and XGBoost using validation and held-out test splits. The current artifact is XGBoost because it achieved the highest recorded validation precision and the strongest overall comparison for the selected metric. The bundle stores the model, feature names, risk thresholds, dataset summary, selection metric, model version, and dataset version.

`artifacts/phishing_detection_model.joblib`

`artifacts/training_report.json`

4.3 Feature Schema

The model currently uses 24 features:

Feature	Purpose
`URLLength`, `DomainLength`	Capture URL and registrable-domain size
`NoOfSpecialCharsInURL`	Measures structural punctuation in the URL
`IsDomainIP`	Detects direct IP-address destinations
`NoOfSubDomain`	Measures subdomain depth
`HasSuspiciousPattern`, `SuspiciousPatternCount`	Capture suspicious keywords and structural patterns
`BrandSimilarityScore`	Measures similarity between the observed label and monitored brand labels
`HasBrandImpersonation`	Indicates a likely lookalike or impersonation relationship
`IsKnownOfficialDomain`, `IsKnownOfficialSubdomain`	Identify configured official domains and trusted subdomains
`BrandInDeceptivePosition`	Detects a brand token outside its official ownership position
`HasAtSymbol`	Detects user-information destination obfuscation using `@`
`IsShorteningService`	Identifies known URL-shortening domains
`HasSensitiveKeyword`, `SensitiveKeywordCount`	Capture account, login, security, payment, and related terms
`HasLongSensitivePath`	Detects long paths containing sensitive terms
`UsesHTTPS`	Records whether HTTPS is used
`NoOfPasswordFields`, `HasLoginForm`	Capture credential collection indicators in HTML
`HasExternalFormSubmit`	Detects forms submitting to another host
`NoOfIFrames`	Counts embedded frames
`DomainTitleMatch`, `TitleDomainTokenOverlap`	Measure page-title consistency with the domain

The feature set combines URL structure, brand identity, sensitive lexical context, and webpage evidence. Login or security words alone are not treated as conclusive phishing evidence. The official-domain catalogue includes commonly used services such as Google, GitHub, GitLab, Microsoft, Monash University, Baidu, and other configured domestic and international services. Official-domain recognition and structural guardrails are used to reduce false positives for legitimate account portals while retaining independent phishing signals such as deceptive brand placement, character substitution, IP hosting, external form submission, or `@`-based destination obfuscation.

4.4 Prediction and Risk Guardrails

The model produces a phishing probability. The stored risk score is adjusted through three groups of guardrails:

1. high-risk guardrails prevent a high probability from directly becoming High Risk when independent phishing evidence is absent
2. missing-web-evidence guardrails avoid treating an unreachable page as confirmed phishing, while preserving strong structural signals
3. benign-evidence guardrails reduce risk for HTTPS pages with consistent domain identity, trusted official identity, and no independent phishing signals

The model thresholds recorded in the active bundle are:

- suspicious threshold: `0.45`
- high-risk threshold: `0.90`

An administrator policy match is evaluated before model inference. An allowlist match produces a Safe result with a policy override version, while a blocklist match produces a High Risk result with the corresponding blocklist override version.

4.5 Explainability

The explanation path is request-specific. The model service extracts local XGBoost tree contributions using `get_booster().predict(..., pred_contribs=True)` when available. The feature-impact adapter combines these values with global feature importance and the actual extracted feature values. The explanation service then selects and formats reasons from the current feature evidence, such as character substitution, deceptive brand placement, an `@` destination, a shortening service, external form submission, or weak domain-title consistency.

The API exposes an explanation mode. The normal XGBoost mode is `tree-shap-local-contributions`; a fallback mode is used if local contributions are unavailable. Policy decisions use `policy-override` and explain that an administrator-reviewed decision replaced ordinary model inference. This makes the provenance of the displayed result explicit.

4.6 Recorded Test Metrics

The current held-out test metrics for the selected XGBoost model are:

Metric	Value
Accuracy	0.9957002826
Precision	0.9947271289
Recall	0.9865341875
F1-score	0.9906137184
ROC-AUC	0.9994473979

These values describe performance on the prepared held-out dataset and should not be interpreted as a guarantee for every live website. Live results also depend on URL novelty, webpage accessibility, redirects, official-domain configuration, and policy overrides.

5 Backend Design

5.1 Application Entry Point

The backend starts from `app/main.py`. Routes are registered for authentication, analysis, reports, and administration. CORS and the development proxy support local frontend-backend testing. The API layer parses requests, checks authentication and roles, converts service errors into structured JSON, and delegates business logic to services.

5.2 API Endpoints

5.2.1 Authentication

- `GET /api/auth/captcha`
- `POST /api/auth/login`
- `POST /api/auth/register`
- `POST /api/auth/change-password`
- `POST /api/auth/reset-password`
- `GET /api/auth/session`
- `POST /api/auth/logout`

5.2.2 Employee Services

- `POST /api/analyze`
- `POST /api/reports`

5.2.3 Administration Services

- `GET /api/admin/overview`
- `GET /api/admin/users`
- `PATCH /api/admin/users/{user_id}/status`
- `GET /api/admin/cases`
- `POST /api/admin/cases/action`
- `GET /api/admin/scans`
- `GET /api/admin/models`
- `GET /api/admin/model/metrics`
- `GET /api/admin/evaluations`
- `GET /api/admin/feedback-dataset`
- `GET /api/admin/policy-entries`
- `DELETE /api/admin/policy-entries/{entry_id}`
- `GET /api/admin/brand-impersonation`
- `GET /api/admin/review-insights`

The list endpoints accept pagination and filtering query parameters. Cases default to ten items per page, scans to forty, and policy, brand, model, and account lists to their configured administrative page sizes.

5.3 Response Contract

An ordinary analysis response contains:

- `requestId`
- `preprocessing`
- `featureExtraction`
- `prediction`
- `verification`
- `explainability`
- `frontend`
- top-level `reasons` and `recommendations`

The `frontend` object contains the calculated safety score, category, verification status, reasons, and recommendations. The employee interface renders the category, verification status, reasons, and recommendations rather than the numeric score. Detailed fields remain available for debugging, logging, administrative displays, and acceptance testing.

5.4 Error Handling

The backend returns structured errors for invalid JSON, invalid URL fields, unsupported schemes, invalid redirect data, missing sessions, insufficient roles, CAPTCHA failures, invalid recovery codes, missing model artifacts, feature-extraction errors, and persistence failures. This allows the frontend to show controlled error states rather than treating all failures as an unavailable service.

6 Persistence Design

6.1 Database Layout

The implementation uses separate SQLite files:

- `auth.sqlite3`: users, roles, password hashes, recovery-code hashes, account status, and sessions
- `prediction_logs.sqlite3`: every completed analysis, model version, risk label, probability, score, and response time
- `clearurl-feedback.sqlite3`: analyses used for feedback, reports, aggregated cases, administrator decisions, and policy entries
- `model_records.sqlite3`: training reports, model versions, dataset versions, and evaluation metrics

This separation avoids forcing authentication, operational logs, feedback workflows, and model evaluation to share one evolving schema. Services locate the database they require through application configuration.

6.2 Feedback and Policy Data

Feedback cases preserve the submitted URL, model version, model result, employee assessment, review type, reason code, comment, report count, and timestamps. Administrator resolution adds the decision, reason, screenshot metadata, deciding account, and optional allowlist expiry.

Policy lookup uses the normalised URL and checks expiry for allowlist entries. The policy result is returned in the analysis response and written to the prediction log with a policy model version. Removing a policy entry does not delete the historical case or scan record.

7 Frontend Design

7.1 Role-Based User Experience

The employee interface is focused on one short task: submit a URL, understand the result, and report or question it when necessary. The admin interface uses a green themed sidebar and a separate workspace so monitoring functions can be selected without mixing employee and administrator actions.

Risk presentation uses green for Safe, amber for Suspicious, and red for High Risk. The employee result displays a category and verification panel so the user can distinguish a lower-risk assessment from a result with limited live evidence without relying on a numeric score.

7.2 Authentication Interface

The authentication screen provides Sign in and Register modes. Register mode exposes Employee and Admin role selection, a CAPTCHA image, and a confirmation password field. Sign-in mode provides Change password and Forgot password actions. Recovery codes are shown in a modal after registration or first recovery-code provisioning and are not written to browser local storage.

7.3 Administration Interface

The administration console includes:

- overview metrics and visual risk distribution
- review cases with status and employee-review context
- scan records with model source and safety-score filters
- a brand visualiser for observed lookalike domains, target brands, similarity, and substitution techniques
- policy list management for allowlist and blocklist entries
- model quality monitoring with precision, recall, F1, ROC-AUC, false-positive rate, false-negative rate, and live operational counts
- account management with role, activity, status, and employee suspension

Large data areas use scrollable panels and server-side pagination. Filter changes reset the corresponding page to the first page and the backend calculates pagination after filtering.

7.4 Feedback Dataset Preparation

Reviewed feedback is not silently added to the training set. The administration endpoint first prepares a tabular feedback dataset that retains the original case, report, model, feature-schema, reason, reviewer-label, and eligibility fields. It can optionally export only reviewed records eligible for training to `data/feedback_dataset.csv`. This provides a traceable hand-off between operational review and any later retraining decision, rather than treating every employee report as an automatically verified label.

8 Testing and Validation

8.1 Automated Validation

The project provides:

- `pytest -q` for backend regression tests
- `npm run build` for TypeScript checking and the production frontend build

The test suite covers authentication and CAPTCHA flows, recovery-code password operations, URL preprocessing, model training behaviour, pipeline response structure, explainability integration, documentation routing, report persistence, policy decisions, pagination, account management, and administrative data APIs.

8.2 Manual and Recorded Evidence

The `docs/` directory contains acceptance criteria, model evaluation records, multiple test records including the final system test record, user stories, task cards, a run sheet, and a GitLab upload allocation record. These records provide traceable evidence for the implemented system.

8.3 Regression Coverage and Validation Boundaries

The automated tests cover URL default-scheme handling, invalid-scheme rejection, official-domain and subdomain recognition, brand impersonation, digit-to-letter substitutions, `@` obfuscation, shortening services, web-fetch outcomes, allowlist and blocklist overrides, allowlist expiry, report-assessment validation, account registration, CAPTCHA, recovery codes, account suspension, administration data, and documentation availability.

Model metrics are measured on prepared dataset splits and do not guarantee correct classification for every live domain. Manual tests must consider official domains, brand substitutions, sensitive keywords on legitimate sites, inaccessible domains, URL shorteners, redirect chains, and policy overrides. Explanations should be checked for correspondence with extracted signals rather than only checking the final category.

9 Deployment and Runtime

9.1 Local Runtime

- frontend: Node.js 22 or later
- backend: Python 3.10 recommended
- backend command: `uvicorn app.main:app --host 127.0.0.1 --port 8000`
- frontend command: `npm run dev`

9.2 Hosted Runtime

For the ECS deployment, the backend runs as a managed service and the frontend is served through the web server and reverse proxy. Updating the application requires stopping or replacing the service safely, synchronising code and runtime assets, rebuilding the frontend when required, restarting the backend, and checking both local and public endpoints.

The deployment must include:

- the Python application and installed backend dependencies
- the production frontend build

- `artifacts/phishing_detection_model.joblib`
- the required SQLite files and writable data directory
- reverse-proxy routing for the frontend and `/api` requests

9.3 Operational Notes

Webpage retrieval depends on network access, DNS resolution, TLS negotiation, remote robots or anti-bot controls, and the target website's response. A failed fetch is represented through verification status and should not automatically be interpreted as proof of maliciousness. Official-domain recognition and policy decisions provide additional context but do not replace ongoing model validation.

10 Limitations and Future Improvements

Current limitations include:

- some live websites block automated HTML retrieval
- an inaccessible or nonexistent domain cannot be confirmed as malicious from availability alone
- official-domain recognition depends on the configured brand and subdomain catalogue
- the safety score is an API and administrative inverse of risk, not a separately trained safety model
- policy overrides are administrator decisions and do not automatically retrain the model
- local SQLite files are suitable for the prototype but would require migration to a managed database for larger production workloads

Future work may include managed database deployment, stronger domain reputation integration, scheduled policy expiry notifications, automated retraining approval, richer user scan history, and additional calibration of safety scores on live validation data.

11 Conclusion

ClearURL is a complete full-stack prototype with a connected employee workflow, role-based administration, trained XGBoost inference, webpage-aware feature extraction, request-specific explanation support, persistent operational records, and administrator-reviewed policy control. The system combines account security, feedback, explainability, monitoring, and scalable administration features into a deployable phishing URL detection service.